Tarlac State University
College of Computer Studies
Bachelor of Science in Computer Science
Web Programming

Case Study: Simulation of CPU Scheduling Algorithms

A Project

Submitted to the Faculty of the College of Computer Studies

San Isidro Campus, Tarlac City,

Tarlac State University

In Partial Fulfillment of the Requirements for the Course Web Programming

Academic Year 2025-2026

Submitted By:
Lamigo, John Eduard P.

Jo Anne Cura
Subject Instructor
Date of submission: (May 15, 2026)

Table of Contents

1. Introduction

CPU scheduling is the mechanism by which an operating system decides the order in which active processes access the CPU. Since there is only one CPU (or a limited number of CPUs), CPU scheduling helps coordinate which process gets CPU access at any given moment. This ensures that multiple tasks can proceed smoothly, minimizing idle time and avoiding bottlenecks (Goyal, 2024).

CPU scheduling is the task performed by the CPU that decides the way and order in which processes should be executed. There are two types of CPU scheduling - Preemptive, and non-preemptive. The criteria the CPU takes into consideration while "scheduling" these processes are - CPU utilization, throughput, turnaround time, waiting time, and response time (Mathur, 2022).

1.2 Purpose of the Study

This study aims to create and execute a CPU scheduling program that will illustrate the functioning of various scheduling algorithms within an operating system. The research seeks to enhance comprehension by illustrating how the algorithms performed on the created programs. This enables users to view each algorithm and its impact on process execution and system performance.

Furthermore, the research aims to assess the effectiveness of every algorithm regarding perform ance measurements. By visualizing the execution of processes with a Gantt chart and the calculated process metrics, this can aid users in grasping how different operating systems handle multiple processes vying for CPU time

1.3 Objectives of the Program

- To simulate and understand the different kinds of CPU scheduling algorithm
- To allow the user visualize the process execution using gantt charts
- To compare the performance of scheduling algorithms based on Waiting Time and Turnaround Time

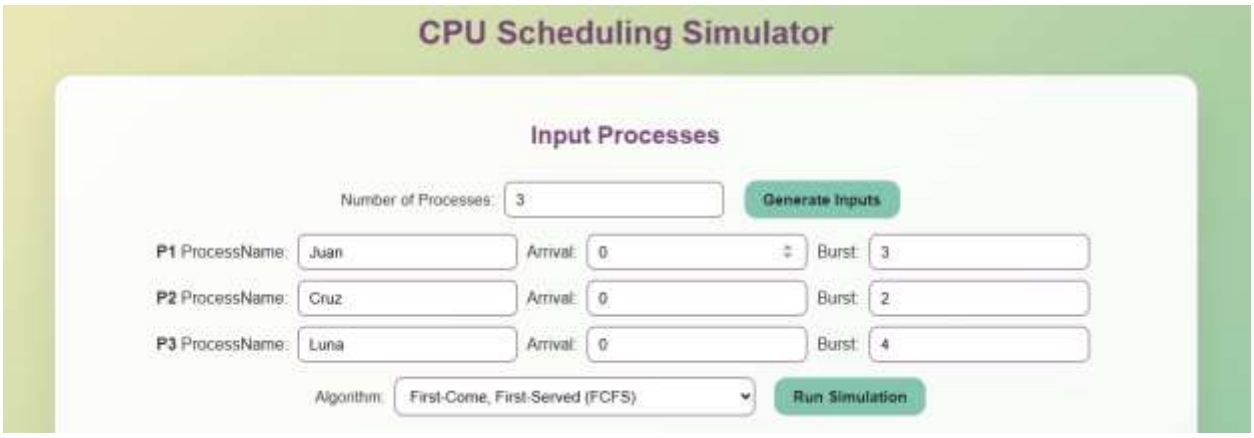
2. Body

2.1 First-Come, First-Served (FCFS)

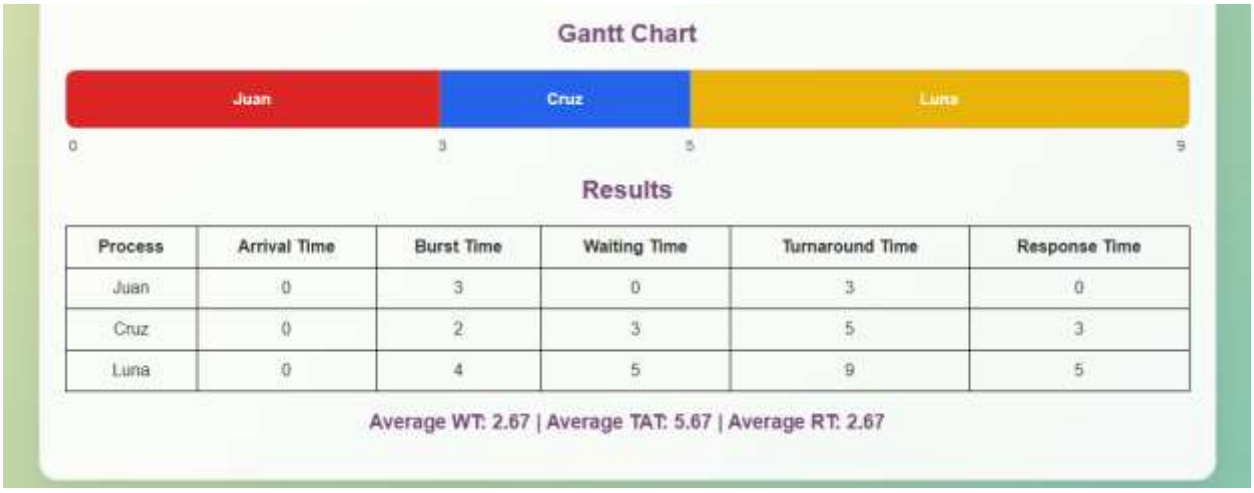
First Come First Serve (FCFS) is one of the easiest CPU Scheduling Algorithms. In FCFS Algorithm, the first process that comes in the ready state queue executes first till its completion. FCFS algorithm employs a non-preemptive strategy, meaning once the process execution begins, no other process can interrupt it. The primary advantage of FCFS is that it is simple and easily implemented. The major disadvantage is the convoy effect, in which the system experiences a long wait time for shorter processes if a lengthy process comes in the ready queue.

Test Case#1

INPUT

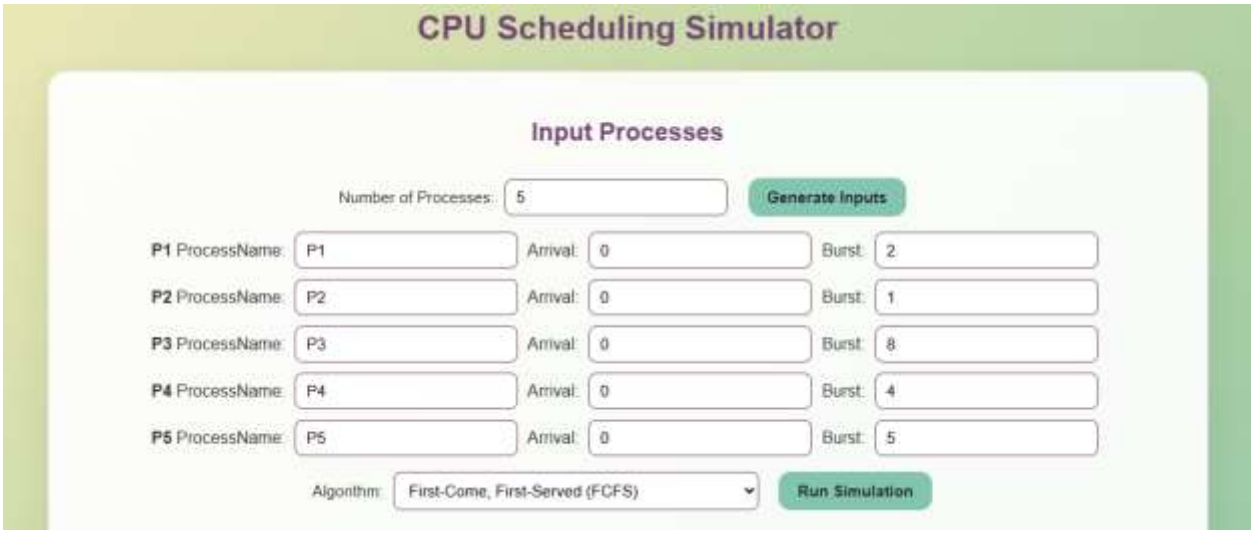


PROCESS METRICS AND GANTT CHART

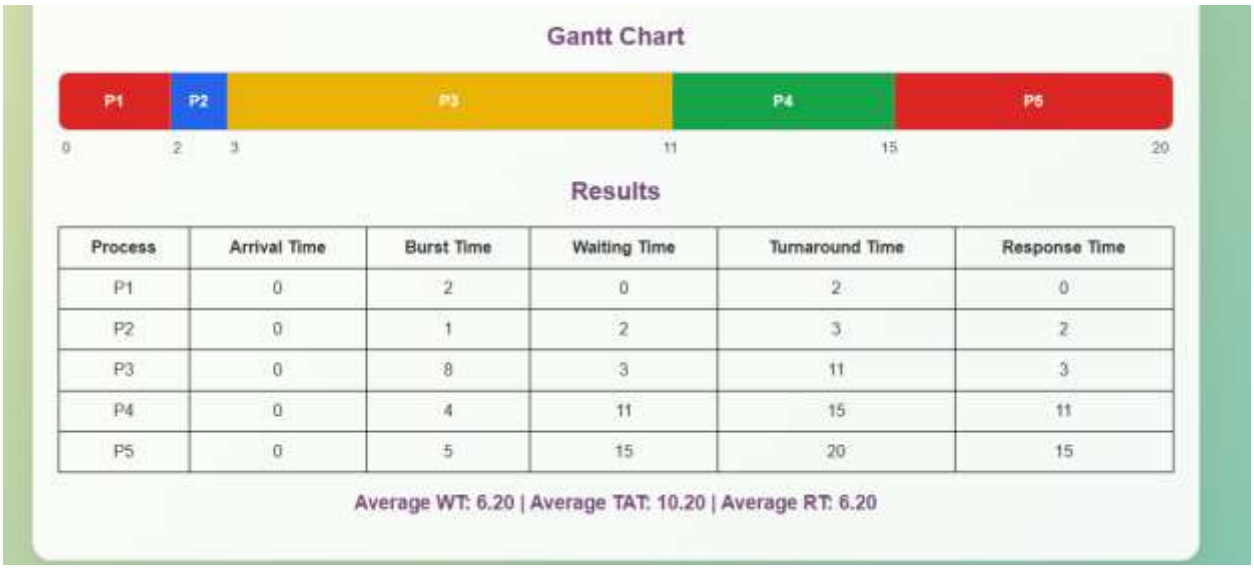


Test Case#2

INPUT



PROCESS METRICS AND GANTT CHART

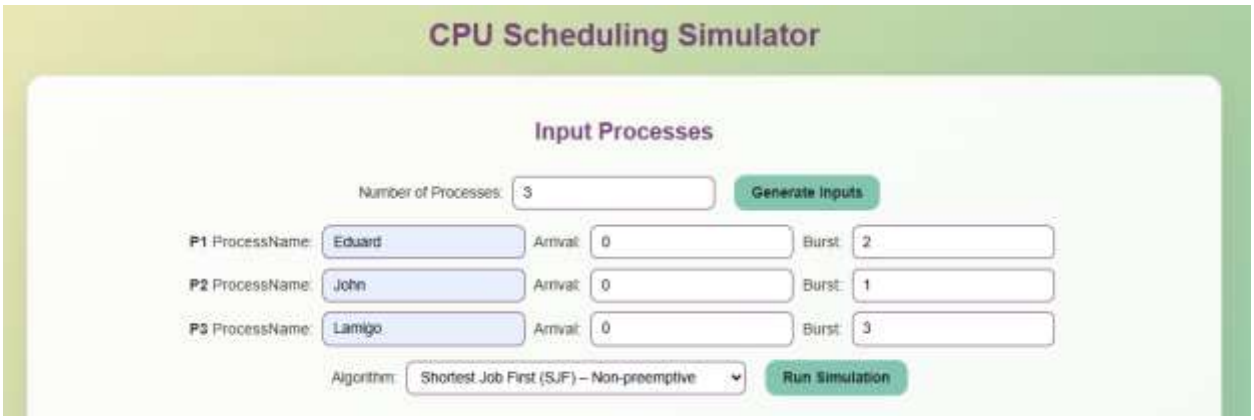


2.2 Shortest Job First (SJF)

Shortest Job First (SJF) is a CPU scheduling algorithm used to pick the shortest job that should be executed first. It is non-preemptive since when the job starts executing, it continues till completion. It is deemed efficient in terms of minimizing the total average waiting time among all processes. The process with shorter waiting time is executed first, thus making the system more efficient. Its weakness is that it can result in starvation. This occurs when there are always jobs coming to execute before the long processes get their chance.

Test Case#1

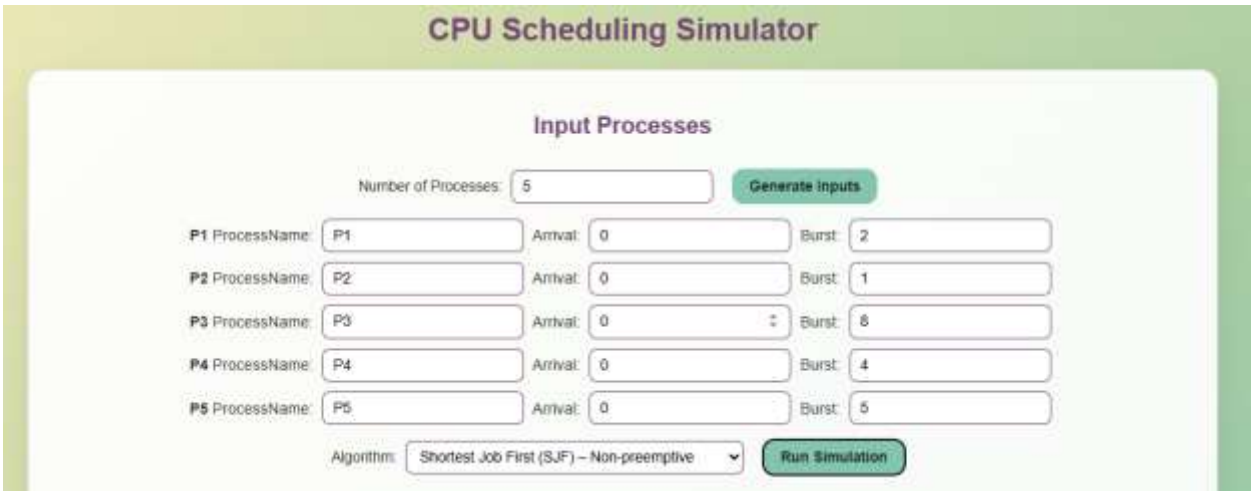
INPUT



PROCESS METRICS AND GANTT CHART

Test Case#2

INPUT



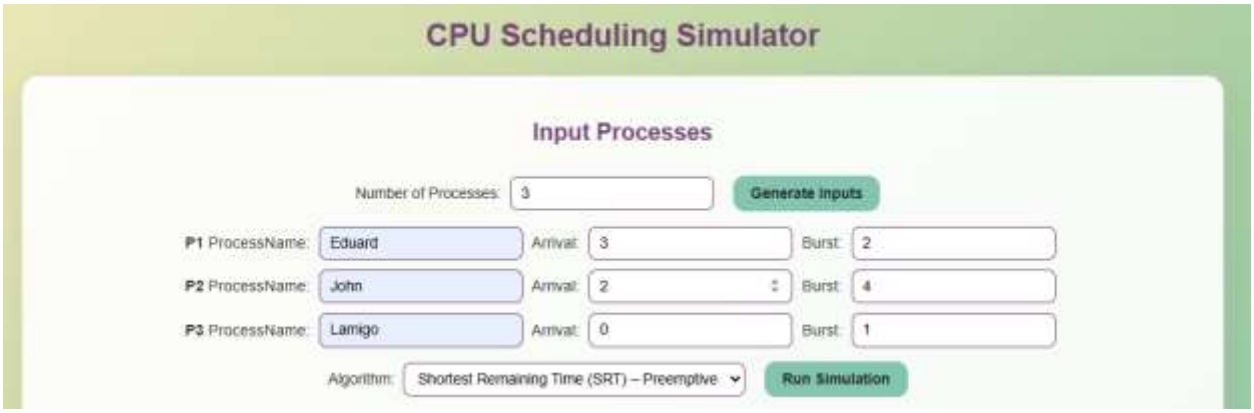
PROCESS METRICS AND GANTT CHART

2.3 Shortest Remaining Time (SRT)

The pre-emptive variant of the shortest job first technique is known as Shortest Remaining Time. Here, at any time, the shortest process in terms of remaining burst time is chosen for execution. In case a process having shorter burst time comes compared to the process running on the processor, then control is immediately transferred from one process to another. Although this technique helps to reduce response time and waiting time, it involves many changes of context.

Test Case#1

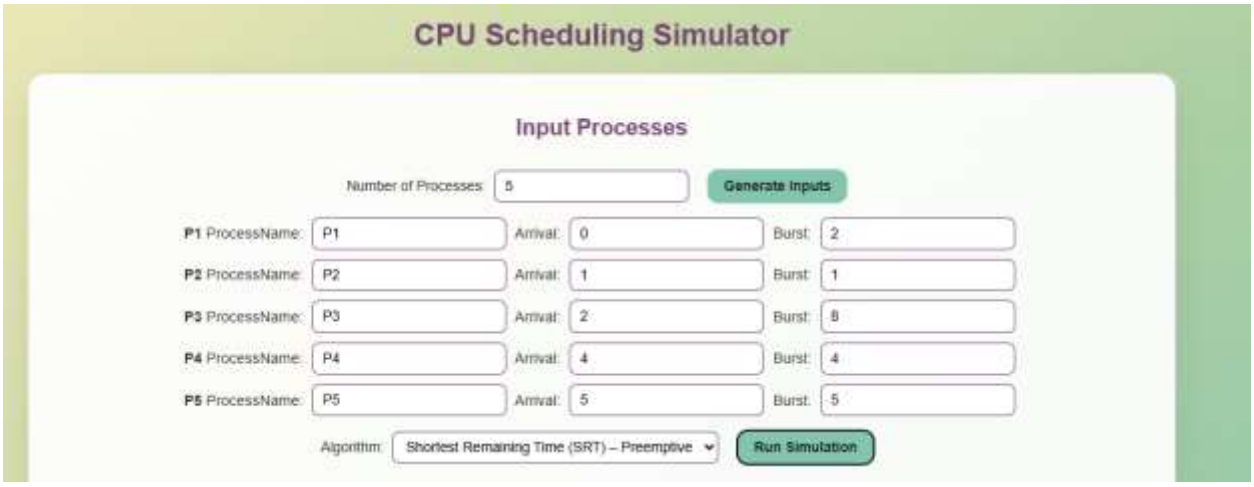
INPUT



PROCESS METRICS AND GANTT CHART

Test Case#2

INPUT



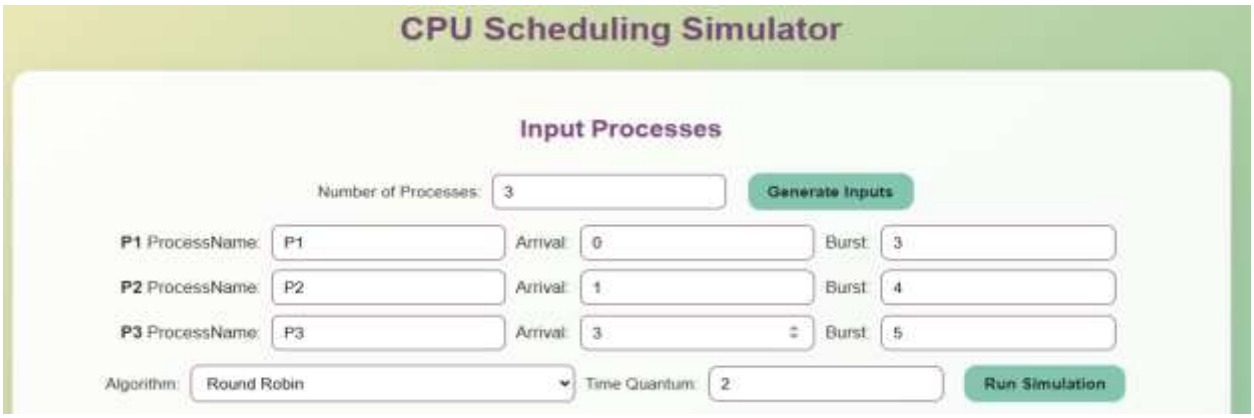
PROCESS METRICS AND GANTT CHART

2.4 Round Robin (RR)

Round Robin (RR) is an algorithm of CPU scheduling based on preemption that is used in time-sharing systems. The CPU is allocated a certain amount of time called time quantum or time slice for each process. After using this time, the CPU moves to the next process in the queue, if the process is not executed entirely within this time, then it is put again at the end of the ready queue. This continues until all processes are executed. Round Robin is considered fair because each process receives the same amount of time on the CPU. But its efficiency greatly depends on the time quantum selected.

Test Case#1

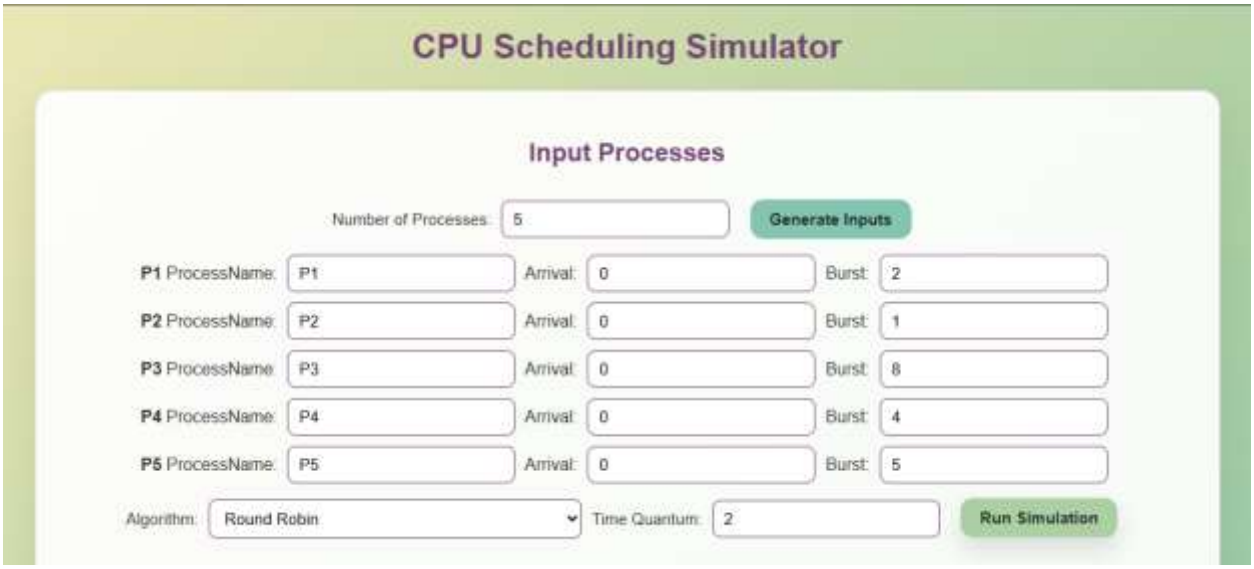
INPUT



PROCESS METRICS AND GANTT CHART

Test Case#2

INPUT



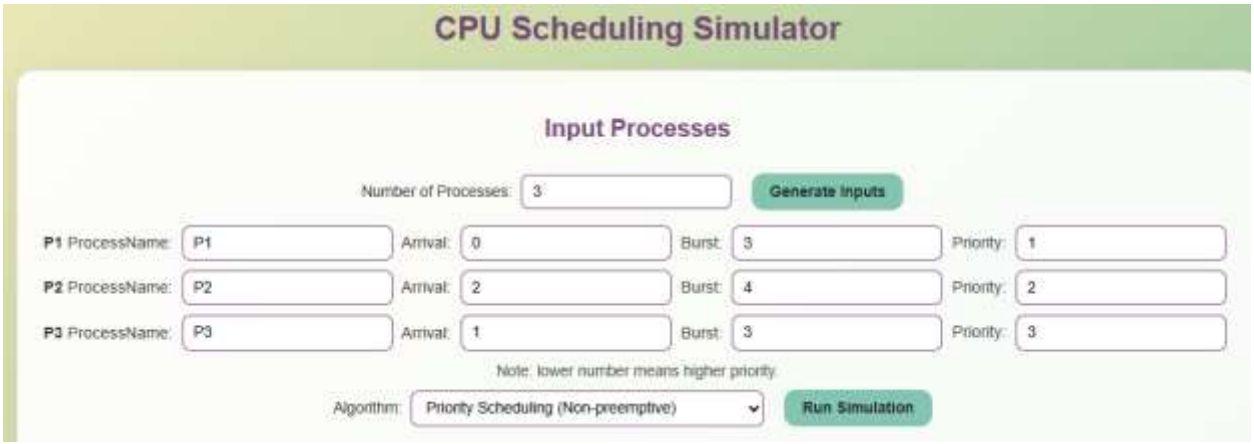
PROCESS METRICS AND GANTT CHART

2.5 Priority Scheduling

The Priority Scheduling algorithm refers to a CPU Scheduling Algorithm wherein the processes are scheduled for execution according to the priority values that are assigned to them. The process that has the highest priority value will execute first in the process queue. In this particular algorithm, the priority value will be such that the smaller the value, the higher the priority value. Priority Scheduling may be preemptive or non-preemptive in nature, but for this case study, the non-preemptive scheduling is used.

Test Case#1

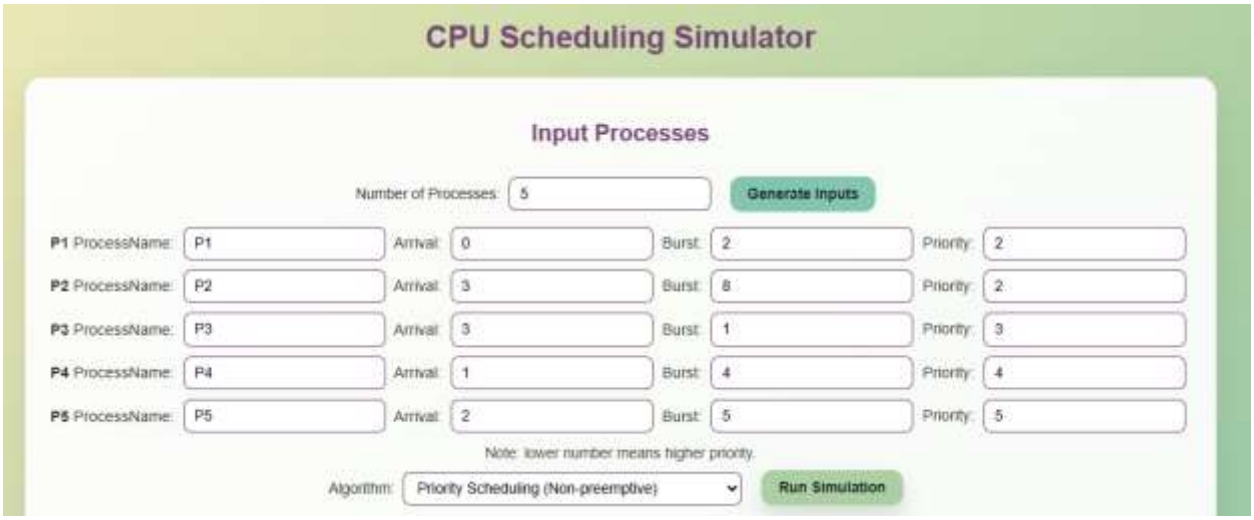
INPUT



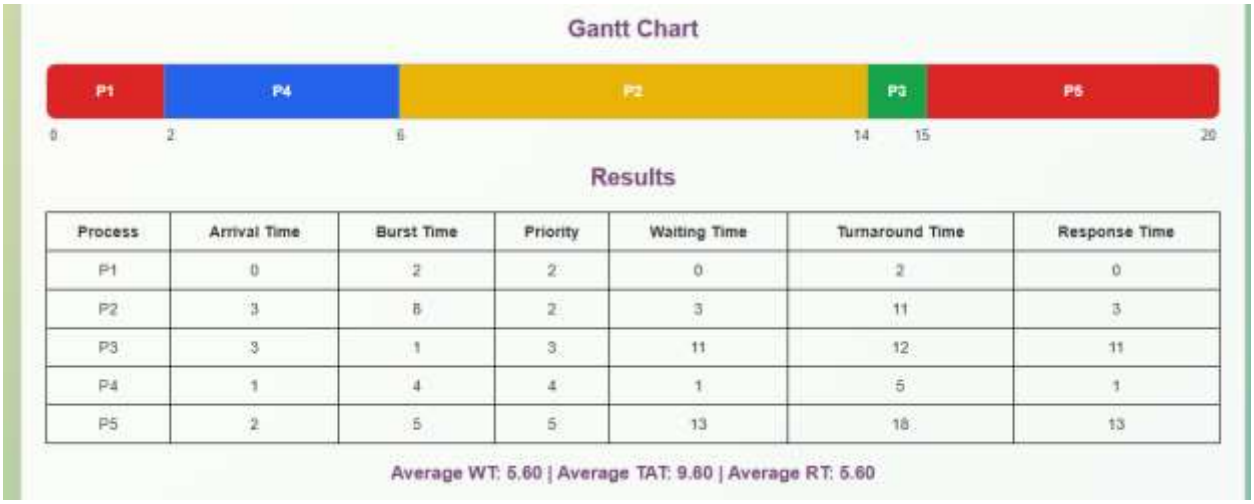
PROCESS METRICS AND GANTT CHART

Test Case#2

INPUT



PROCESS METRICS AND GANTT CHART

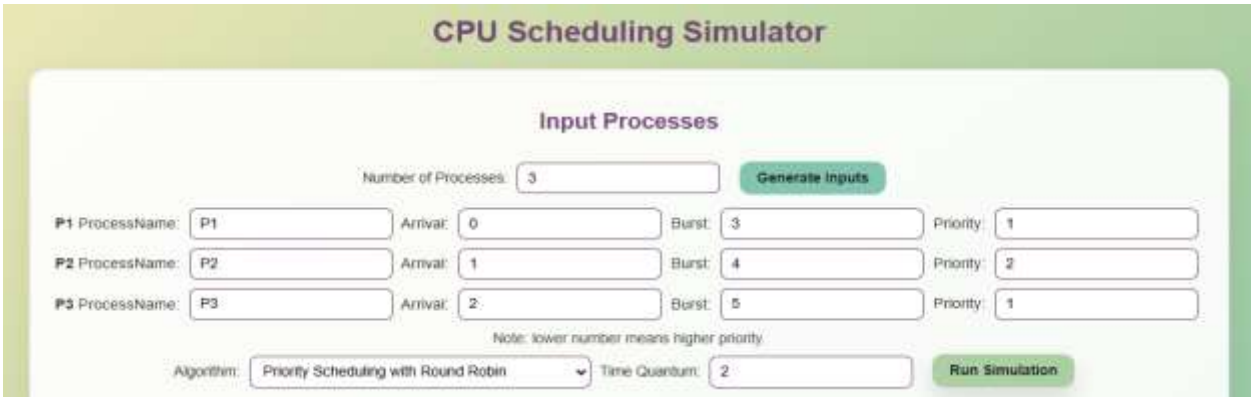


2.6 Priority Scheduling with Round Robin

Combination of Priority Scheduling with Round Robin makes use of Priority Scheduling and Round Robin Scheduling techniques. Here, the processes will be divided depending upon their priority level, and those processes having similar priorities are scheduled using the Round Robin approach. Thus, this technique ensures equality and also maintains high priority. The method follows the preemption approach as the processes will be preempted on the basis of the time quantum and priorities. While providing better scheduling, this method is comparatively difficult to implement than other methods as it involves priority as well as round robin handling**.**

Test Case#1

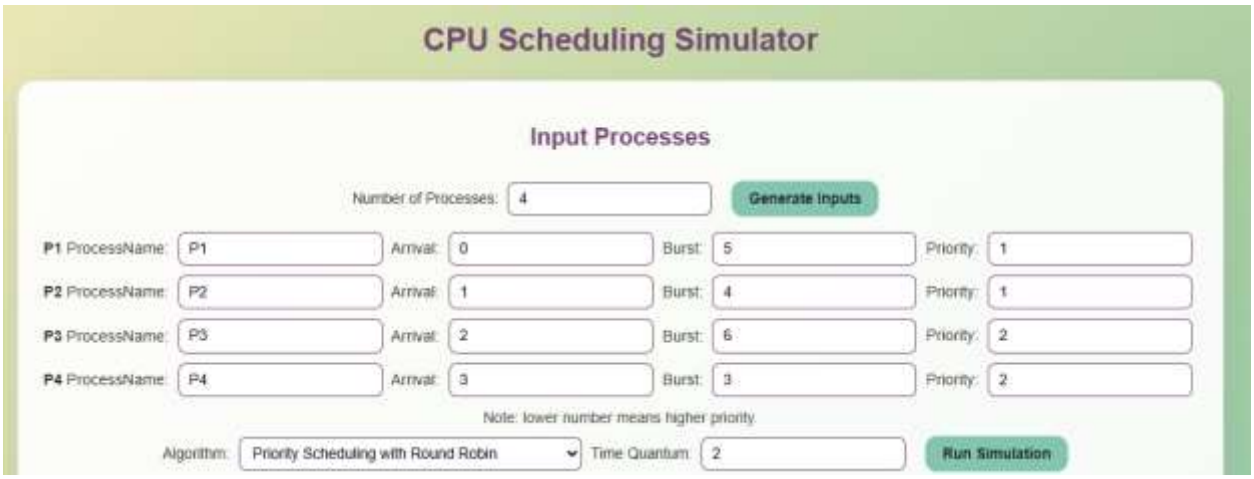
INPUT



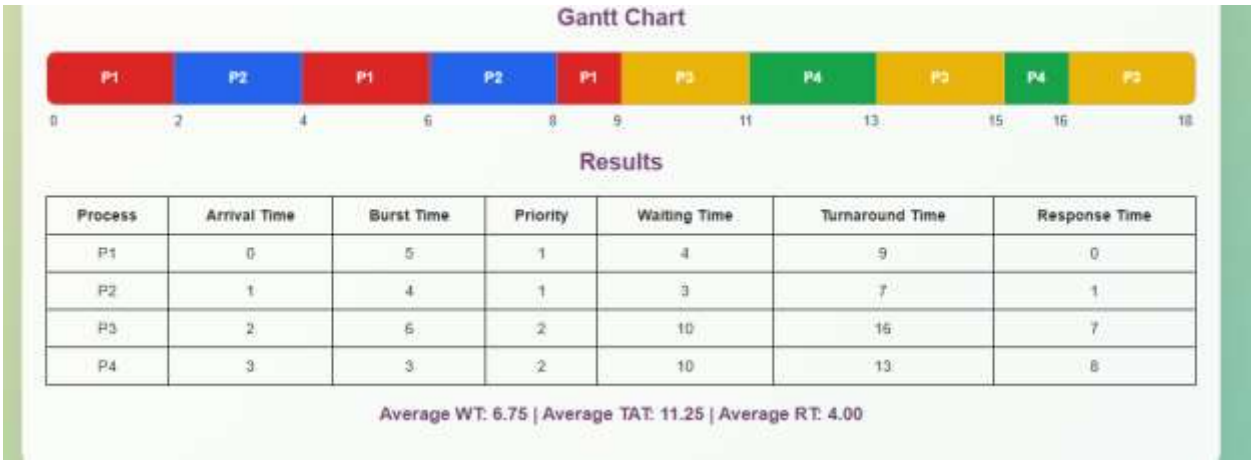
PROCESS METRICS AND GANTT CHART

Test Case#2

INPUT



PROCESS METRICS AND GANTT CHART



3. Summary / Takeaways

3.1 Key Insights

In conclusion, this case study gave me an insight into how CPU Scheduling Algorithms function in controlling processes in an Operating System. Different scheduling algorithms helped me understand various principles like process execution sequence, CPU usage, waiting time, turnaround time, and process priority. This project was also helpful in sharpening my programming and analysis abilities since every algorithm needed different approaches to its implementation. From this simulation, it is clear that scheduling algorithms have an effect on the performance of a computer system.

3.2 Performance Comparison Observations

The various scheduling methods differed considerably regarding fairness, efficiency, and waiting period. First Come First Serve was straightforward to use and implement, yet it occasionally resulted in prolonged waiting periods since shorter jobs had to wait behind longer ones. Shortest Job First and Shortest Remaining Time ensured that the average waiting period was reduced and were efficient; however, longer jobs could starve. Round Robin was fairer since each job received equal CPU usage due to the allocation of time quantum. Priority Scheduling emphasized executing crucial processes, while Priority Scheduling with Round Robin was fairer and managed priorities.

3.3 Practical Applications in Operating Systems

The role of CPU scheduling is critical within current operating systems because it makes it possible to manage different processes more effectively. Scheduling algorithms make it possible to optimize CPU usage, minimize delay times, and ensure that the system remains responsive. It is critical within multi-tasking environments to schedule correctly so that applications and

background tasks continue to function without experiencing significant disruption. Such algorithms are widely utilized in current computers and servers.

3.4 Challenges Encountered and Solutions

There were various obstacles experienced in implementing the scheduling algorithms. One of the major issues involved in generating the right Gantt charts, particularly for the pre-emptive algorithm. Due to interruption and resumption of execution many times by various processes, making the right execution sequence and timing became difficult. Other challenges experienced involved managing the quantum value for Round Robin and Priority Scheduling with Round Robin, where any mistake made on the management of the quantum value would give the wrong process sequences and thus lead to an erroneous calculation of waiting and turnaround times.

This problem was overcome by going through the whole execution process and observing the time remaining for each process after each process completion. This was also assisted by manual checks on the timeline of the Gantt Chart and further tests performed on various sample input cases.

4.References

Goyal, S. (2024, February 16). *CPU Scheduling In Operating Systems | All Concepts Explained*. Unstop.com; Unstop. https://unstop.com/blog/cpu-scheduling-in-operating-system

Mathur, T. (2022, April 28). *CPU scheduling*. Scaler Topics; Scaler Topics. https://www.scaler.com/topics/operating-system/cpu-scheduling/

Main Program

```
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="UTF-8">

    <title>CPU Scheduling Simulator</title>

    <link rel="stylesheet" href="styles.css">

</head>

<body>


<h1>CPU Scheduling Simulator</h1>


<div class="container">


    <h2>Input Processes</h2>

    <!-- User input area for process parameters and algorithm selection -->

    <label>Number of Processes:</label>

    <input type="number" id="numProcesses" min="3" value="3">


    <button onclick="generateInputs()">Generate Inputs</button>


    <div id="processInputs"></div>

    <div id="priorityNote" style="display:none; font-size:0.95rem; color:#444; margin-top:8px;">

        Note: lower number means higher priority.

    </div>


    <label>Algorithm:</label>

    <select id="algorithm" onchange="updateVisibility()">

        <option value="fcfs">First-Come, First-Served (FCFS)</option>

        <option value="sjf">Shortest Job First (SJF) – Non-preemptive</option>

        <option value="srt">Shortest Remaining Time (SRT) – Preemptive </option>

        <option value="rr">Round Robin</option>

        <option value="prio">Priority Scheduling (Non-preemptive)</option>

        <option value="prio_rr">Priority Scheduling with Round Robin</option>

    </select>


    <label id="quantumLabel" style="display:none;">Time Quantum:</label>

    <input type="number" id="quantum" min="1" value="2" style="display:none;">


    <button onclick="runSimulation()">Run Simulation</button>


    <h2>Gantt Chart</h2>

    <div id="ganttContainer">

        <div id="gantt"></div>

        <div id="ganttLabels"></div>

    </div>


    <h2>Results</h2>

    <table id="resultTable">

        <thead id="tableHeader">

            <tr>

                <th>Process</th>

                <th>Arrival Time</th>

                <th>Burst Time</th>

                <th id="priorityHeader">Priority</th>

                <th>Waiting Time</th>

                <th>Turnaround Time</th>

                <th>Response Time</th>
```

```
          </tr>
        </thead>
        <tbody></tbody>
    </table>

    <h3 id="averages"></h3>

</div>
<script src="utils.js"></script>
<script src="fcfs.js"></script>
<script src="sjf.js"></script>
<script src="srt.js"></script>
<script src="rr.js"></script>
<script src="priority.js"></script>
<script src="priority_rr.js"></script>
<script src="main.js"></script>
</body>
</html>
```

CCS Style

```
body {
    font-family: Arial, sans-serif;
    background: linear-gradient(135deg, #F0E9B6 0%, #ACCFA3 60%, #84C5B1 100%);
    text-align: center;
    color: #2d2d2d;
}

.container {
    background: rgba(255, 255, 255, 0.92);
    padding: 24px;
    margin: auto;
    width: 80%;
    border-radius: 18px;
    border: 1px solid rgba(116, 69, 119, 0.18);
    box-shadow: 0 18px 40px rgba(0, 0, 0, 0.08);
}

h1, h2, h3 {
    color: #744577;
}

label, select, input, button {
    font-size: 0.95rem;
}

input, select {
    margin: 5px;
    border: 1px solid #744577;
    border-radius: 8px;
    padding: 8px 10px;
}

button {
    padding: 10px 18px;
    margin: 10px;
    cursor: pointer;
    border: none;
    border-radius: 12px;
    background: #84C5B1;
    color: #1f2a27;
    font-weight: 700;
    transition: transform 0.2s ease, box-shadow 0.2s ease, background 0.2s ease;
}

button:hover {
    background: #ACCFA3;
    transform: translateY(-1px);
    box-shadow: 0 12px 22px rgba(0, 0, 0, 0.12);
}

#ganttContainer {
    margin-top: 20px;
}

#gantt {
    display: flex;
    align-items: stretch;
```

```
    min-height: 54px;
    border: 1px solid rgba(116, 69, 119,
0.35);
    border-radius: 10px;
    overflow: hidden;
    background: #f8faf8;
}

.gantt-segment {
    display: flex;
    align-items: center;
    justify-content: center;
    position: relative;
    color: white;
    font-weight: 700;
    min-width: 48px;
    border-right: 1px solid rgba(255, 255,
255, 0.25);
}

.gantt-segment:last-child {
    border-right: none;
}

.gantt-segment .segment-label {
    position: absolute;
    left: 50%;
    transform: translateX(-50%);
    white-space: nowrap;
    font-size: 0.95rem;
}

#ganttLabels {
    display: flex;
    gap: 0;
    margin-top: 8px;
    font-size: 0.9rem;
    color: #334155;
}

#ganttLabels span {
    text-align: left;
    padding-left: 4px;
}

#ganttLabels span:last-child {
    text-align: right;
    padding-right: 4px;
}

table {
    width: 100%;
    margin-top: 20px;
    border-collapse: collapse;
}

table, th, td {
    border: 1px solid black;
}

th, td {
    padding: 10px;
}
```

Main Program's JavaScript

```
// main.js - Main UI logic for CPU
Scheduling Simulator

function generateInputs() {
    let n =
parseInt(document.getElementById("numPr
ocesses").value, 10) || 0;
    if (n < 3) {
        alert("Please enter at least 3
processes.");

document.getElementById("numProcesses")
.value = 3;
        n = 3;
    }
    let container =
document.getElementById("processInputs")
;
    container.innerHTML = "";

    for (let i = 0; i < n; i++) {
        container.innerHTML += `
            <div>
                <b>P${i + 1}</b>
                ProcessName: <input type="text"
id="pn${i}" value="P${i + 1}">
```

```
            Arrival: <input type="number"
id="at${i}" value="0" min="0">
            Burst: <input type="number"
id="bt${i}" value="1" min="0">
            <span id="priority${i}">Priority:
<input type="number" id="pr${i}"
value="${i + 1}" min="0"></span>
        </div>
    `;
  }
  updateVisibility();
}

function updateVisibility() {
  // Show or hide inputs depending on the
selected scheduling algorithm.
  let algo =
document.getElementById("algorithm").val
ue;
  let quantumLabel =
document.getElementById("quantumLabel"
);
  let quantumInput =
document.getElementById("quantum");

  if (algo === "rr" || algo === "prio_rr") {
    quantumLabel.style.display = "inline";
    quantumInput.style.display = "inline";
  } else {
    quantumLabel.style.display = "none";
    quantumInput.style.display = "none";
  }

  // Show/hide priority inputs only for
priority-based algorithms
  let n =
parseInt(document.getElementById("numPr
ocesses").value, 10) || 0;
  let showPriority = (algo === "prio" || algo
=== "prio_rr");

  for (let i = 0; i < n; i++) {
    let prioritySpan =
document.getElementById("priority" + i);
    if (prioritySpan) {
      prioritySpan.style.display =
showPriority ? "inline" : "none";
    }
  }

  let priorityNote =
document.getElementById("priorityNote");
  if (priorityNote) {
    priorityNote.style.display =
showPriority ? "block" : "none";
  }

  // Update table header
  updateTableHeader(algo);
}

function runSimulation() {
  let n =
parseInt(document.getElementById("numPr
ocesses").value, 10) || 0;
  if (n < 3) {
    alert("Please use at least 3 processes
before running the simulation.");
    return;
  }
  let algo =
document.getElementById("algorithm").val
ue;

  for (let i = 0; i < n; i++) {
    let pn =
document.getElementById("pn" +
i).value.trim();
    let at =
parseInt(document.getElementById("at" +
i).value, 10);
    let bt =
parseInt(document.getElementById("bt" +
i).value, 10);
    let prInput =
document.getElementById("pr" + i);
    let pr = prInput ?
parseInt(prInput.value, 10) : 0;

    if (pn === "") {
```

```
            alert(`Please provide a name for
process ${i + 1}.`);
            return;
        }
        if (isNaN(at) || at < 0) {
            alert(`Arrival time for process ${i +
1} must be 0 or higher.`);
            return;
        }
        if (isNaN(bt) || bt < 0) {
            alert(`Burst time for process ${i + 1}
must be 0 or higher.`);
            return;
        }
        if (prInput && (isNaN(pr) || pr < 0)) {
            alert(`Priority for process ${i + 1}
must be 0 or higher.`);
            return;
        }
    }

    if ((algo === "rr" || algo === "prio_rr")) {
        let quantum =
parseInt(document.getElementById("quantu
m").value, 10);
        if (isNaN(quantum) || quantum < 1) {
            alert("Time quantum must be a
positive integer of 1 or higher.");
            return;
        }
    }

    // Run the selected scheduling algorithm
and draw the results.
    if (algo === "fcfs") {
        fcfs();
    } else if (algo === "sjf") {
        sjf();
    } else if (algo === "srt") {
        srt();
    } else if (algo === "rr") {
        rr();
    } else if (algo === "prio") {
        priority();
    } else if (algo === "prio_rr") {
        priorityRR();
    } else {
        alert("Algorithm not implemented
yet.");
    }
}

document.addEventListener("DOMContent
Loaded", generateInputs);
```

First Come First Serve Algorithm

```
function fcfs() {
    let processes = getProcessInputs();
    processes.sort((a, b) => a.at - b.at ||
a.id.localeCompare(b.id));

    let time = 0;
    let schedule = [];
    let wt = [];
    let tat = [];
    let rt = [];

    processes.forEach((p, i) => {
        if (time < p.at) {
            createScheduleSegment(schedule,
'Idle', time, p.at);
            time = p.at;
        }
        p.responseTime = time - p.at;
        p.completionTime = time + p.bt;
        wt[i] = time - p.at;
        tat[i] = p.completionTime - p.at;
        rt[i] = p.responseTime;
        createScheduleSegment(schedule, p.pn,
time, p.completionTime);
        time = p.completionTime;
    });

    displayResults(processes, wt, tat, rt,
schedule);
}
```

Shortest Job First Algorithm

```
function sjf() {
```

```
    let processes = getProcessInputs();
    let time = 0;
    let completed = 0;
    let schedule = [];
    let wt = [];
    let tat = [];
    let rt = [];

    while (completed < processes.length) {
        let ready = processes.filter(p => p.at <=
time && !p.done);
        if (ready.length === 0) {
            let nextArrival =
Math.min(...processes.filter(p =>
!p.done).map(p => p.at));
            createScheduleSegment(schedule,
'Idle', time, nextArrival);
            time = nextArrival;
            continue;
        }

        ready.sort((a, b) => a.bt - b.bt || a.at -
b.at || a.id.localeCompare(b.id));
        let current = ready[0];
        if (current.responseTime === -1) {
            current.responseTime = time -
current.at;
        }
        current.completionTime = time +
current.bt;
        createScheduleSegment(schedule,
current.pn, time, current.completionTime);
        time = current.completionTime;
        current.done = true;
        completed++;
        let index = processes.findIndex(p =>
p.id === current.id);
        wt[index] = current.completionTime -
current.at - current.bt;
        tat[index] = current.completionTime -
current.at;
        rt[index] = current.responseTime;
    }

    displayResults(processes, wt, tat, rt,
schedule);
}
```

Shortest Remaining Time Algorithm

```
function srt() {
    let processes = getProcessInputs();
    let time = 0;
    let completed = 0;
    let schedule = [];
    let wt = [];
    let tat = [];
    let rt = [];

    while (completed < processes.length) {
        let ready = processes.filter(p => p.at <=
time && !p.done);
        if (ready.length === 0) {
            let nextArrival =
Math.min(...processes.filter(p =>
!p.done).map(p => p.at));
            createScheduleSegment(schedule,
'Idle', time, nextArrival);
            time = nextArrival;
            continue;
        }

        ready.sort((a, b) => a.remaining -
b.remaining || a.at - b.at ||
a.id.localeCompare(b.id));
        let current = ready[0];
        if (current.responseTime === -1) {
            current.responseTime = time -
current.at;
        }

        createScheduleSegment(schedule,
current.pn, time, time + 1);
        time += 1;
        current.remaining -= 1;

        if (current.remaining === 0) {
            current.completionTime = time;
            current.done = true;
            completed++;
            let index = processes.findIndex(p =>
p.id === current.id);
```

```
        wt[index] = current.completionTime
- current.at - current.bt;
        tat[index] = current.completionTime
- current.at;
        rt[index] = current.responseTime;
      }
   }

   displayResults(processes, wt, tat, rt,
schedule);
}
```

Round Robin Algorithm

```
function rr() {
   let processes = getProcessInputs();
   let quantum = getTimeQuantum();
   let time = 0;
   let queue = [];
   let schedule = [];
   let wt = [];
   let tat = [];
   let rt = [];
   let completed = 0;
   let index = 0;

   while (completed < processes.length) {
      while (index < processes.length &&
processes[index].at <= time) {
         queue.push(processes[index]);
         index++;
      }

      if (queue.length === 0) {
         let nextArrival =
Math.min(...processes.filter(p =>
!p.done).map(p => p.at));
         createScheduleSegment(schedule,
'Idle', time, nextArrival);
         time = nextArrival;
         continue;
      }

      let current = queue.shift();
      if (current.responseTime === -1) {
         current.responseTime = time -
current.at;
      }

      let executeTime = Math.min(quantum,
current.remaining);
      createScheduleSegment(schedule,
current.pn, time, time + executeTime);
      time += executeTime;
      current.remaining -= executeTime;

      while (index < processes.length &&
processes[index].at <= time) {
         queue.push(processes[index]);
         index++;
      }

      if (current.remaining > 0) {
         queue.push(current);
      } else {
         current.completionTime = time;
         current.done = true;
         completed++;
         let idx = processes.findIndex(p =>
p.id === current.id);
         wt[idx] = current.completionTime -
current.at - current.bt;
         tat[idx] = current.completionTime -
current.at;
         rt[idx] = current.responseTime;
      }
   }

   displayResults(processes, wt, tat, rt,
schedule);
}
```

Priority Algorithm

```
function priority() {
   let processes = getProcessInputs();
   let time = 0;
   let completed = 0;
   let schedule = [];
   let wt = [];
   let tat = [];
   let rt = [];
```

```
    while (completed < processes.length) {
        let ready = processes.filter(p => p.at <=
time && !p.done);
        if (ready.length === 0) {
            let nextArrival =
Math.min(...processes.filter(p =>
!p.done).map(p => p.at));
            createScheduleSegment(schedule,
'Idle', time, nextArrival);
            time = nextArrival;
            continue;
        }

        ready.sort((a, b) => a.pr - b.pr || a.at -
b.at || a.id.localeCompare(b.id));
        let current = ready[0];
        if (current.responseTime === -1) {
            current.responseTime = time -
current.at;
        }
        createScheduleSegment(schedule,
current.pn, time, time + current.bt);
        current.completionTime = time +
current.bt;
        time = current.completionTime;
        current.done = true;
        completed++;
        let index = processes.findIndex(p =>
p.id === current.id);
        wt[index] = current.completionTime -
current.at - current.bt;
        tat[index] = current.completionTime -
current.at;
        rt[index] = current.responseTime;
    }

    displayResults(processes, wt, tat, rt,
schedule);
}
```

Priority with Round Robin Algorithm

```
function priorityRR() {
    let processes = getProcessInputs();
    let quantum = getTimeQuantum();
    let time = 0;
    let queue = [];
    let schedule = [];
    let wt = [];
    let tat = [];
    let rt = [];
    let completed = 0;
    let index = 0;

    processes.sort((a, b) => a.at - b.at ||
a.id.localeCompare(b.id));

    while (completed < processes.length) {
        while (index < processes.length &&
processes[index].at <= time) {
            queue.push(processes[index]);
            index++;
        }

        if (queue.length === 0) {
            let nextArrival =
Math.min(...processes.filter(p =>
!p.done).map(p => p.at));
            createScheduleSegment(schedule,
'Idle', time, nextArrival);
            time = nextArrival;
            continue;
        }

        queue.sort((a, b) => a.pr - b.pr || a.at -
b.at || a.id.localeCompare(b.id));
        let current = queue.shift();
        if (current.responseTime === -1) {
            current.responseTime = time -
current.at;
        }

        let executeTime = Math.min(quantum,
current.remaining);
```

```
        createScheduleSegment(schedule,
current.pn, time, time + executeTime);
        time += executeTime;
        current.remaining -= executeTime;

        while (index < processes.length &&
processes[index].at <= time) {
            queue.push(processes[index]);
            index++;
        }

        if (current.remaining > 0) {
            queue.push(current);
        } else {
            current.completionTime = time;
            current.done = true;
            completed++;
            let idx = processes.findIndex(p =>
p.id === current.id);
            wt[idx] = current.completionTime -
current.at - current.bt;
            tat[idx] = current.completionTime -
current.at;
            rt[idx] = current.responseTime;
        }
    }

    displayResults(processes, wt, tat, rt,
schedule);
}
```

Gantt Chart's JavaScript

```
function createScheduleSegment(schedule,
pid, start, end) {
    // Add or merge a segment into the
schedule, keeping adjacent same-process
blocks together.
    if (start >= end) return;
    if (schedule.length &&
schedule[schedule.length - 1].pid === pid
&& schedule[schedule.length - 1].end ===
start) {
        schedule[schedule.length - 1].end =
end;
    } else {
        schedule.push({ pid, start, end });
    }
}

function getProcessInputs() {
    let n =
parseInt(document.getElementById("numPr
ocesses").value, 10);
    let processes = [];

    for (let i = 0; i < n; i++) {
        processes.push({
            id: "P" + (i + 1),
            pn: document.getElementById("pn"
+ i).value,
            at:
parseInt(document.getElementById("at" +
i).value, 10),
            bt:
parseInt(document.getElementById("bt" +
i).value, 10),
            pr:
parseInt(document.getElementById("pr" +
i).value, 10),
            remaining:
parseInt(document.getElementById("bt" +
i).value, 10),
            completionTime: 0,
            responseTime: -1,
            done: false
        });
    }

    return processes;
}

function getTimeQuantum() {
    return
parseInt(document.getElementById("quantu
m").value, 10);
}
```

```
function updateTableHeader(algo) {
    let priorityHeader =
document.getElementById("priorityHeader"
);
    if (algo === "prio" || algo === "prio_rr")
{
        priorityHeader.style.display = "table-
cell";
    } else {
        priorityHeader.style.display = "none";
    }
}

function displayResults(processes, wt, tat, rt,
schedule) {
    let ganttDiv =
document.getElementById("gantt");
    let labelsDiv =
document.getElementById("ganttLabels");
    ganttDiv.innerHTML = "";
    labelsDiv.innerHTML = "";

    let totalWT = 0;
    let totalTAT = 0;
    let totalRT = 0;
    let n = processes.length;
    let algo =
document.getElementById("algorithm").val
ue;
    let showPriority = (algo === "prio" || algo
=== "prio_rr");

    const palette = ['#dc2626', '#2563eb',
'#eab308', '#16a34a'];
    const colors = {};
    let nextColorIndex = 0;
    schedule.forEach(segment => {
        if (!colors[segment.pid]) {
            colors[segment.pid] = segment.pid
=== 'Idle' ? '#d1d5db' :
palette[nextColorIndex % palette.length];
            nextColorIndex++;
        }
        const block =
document.createElement('div');
        block.className = 'gantt-segment';
        block.style.flex = `${segment.end -
segment.start}`;
        block.style.background =
colors[segment.pid];
        block.innerHTML = `<span
class="segment-
label">${segment.pid}</span>`;
        ganttDiv.appendChild(block);

        const timeLabel =
document.createElement('span');
        timeLabel.style.flex = `${segment.end -
segment.start}`;
        timeLabel.textContent = segment.start;
        labelsDiv.appendChild(timeLabel);
    });

    if (schedule.length > 0) {
        const lastLabel =
document.createElement('span');
        lastLabel.textContent =
schedule[schedule.length - 1].end;
        lastLabel.style.flex = '0';
        labelsDiv.appendChild(lastLabel);
    }

    let tbody =
document.querySelector("#resultTable
tbody");
    tbody.innerHTML = "";

    processes.forEach((p, i) => {
        totalWT += wt[i];
        totalTAT += tat[i];
        totalRT += rt[i];

        let priorityCell = showPriority ?
`<td>${p.pr}</td>` : '';

        tbody.innerHTML += `
            <tr>
                <td>${p.pn}</td>
                <td>${p.at}</td>
                <td>${p.bt}</td>
                ${priorityCell}
```

```
            <td>${wt[i]}</td>
            <td>${tat[i]}</td>
            <td>${rt[i]}</td>
        </tr>
    `;
});

let avgWT = totalWT / n;
let avgTAT = totalTAT / n;
let avgRT = totalRT / n;


document.getElementById("averages").inne
rText =
    `Average WT: ${avgWT.toFixed(2)} |
Average TAT: ${avgTAT.toFixed(2)} |
Average RT: ${avgRT.toFixed(2)}`;
}
```

https://github.com/kagemi198/-CPU-Scheduling-Algorithm-in-Operating-Systems-.git